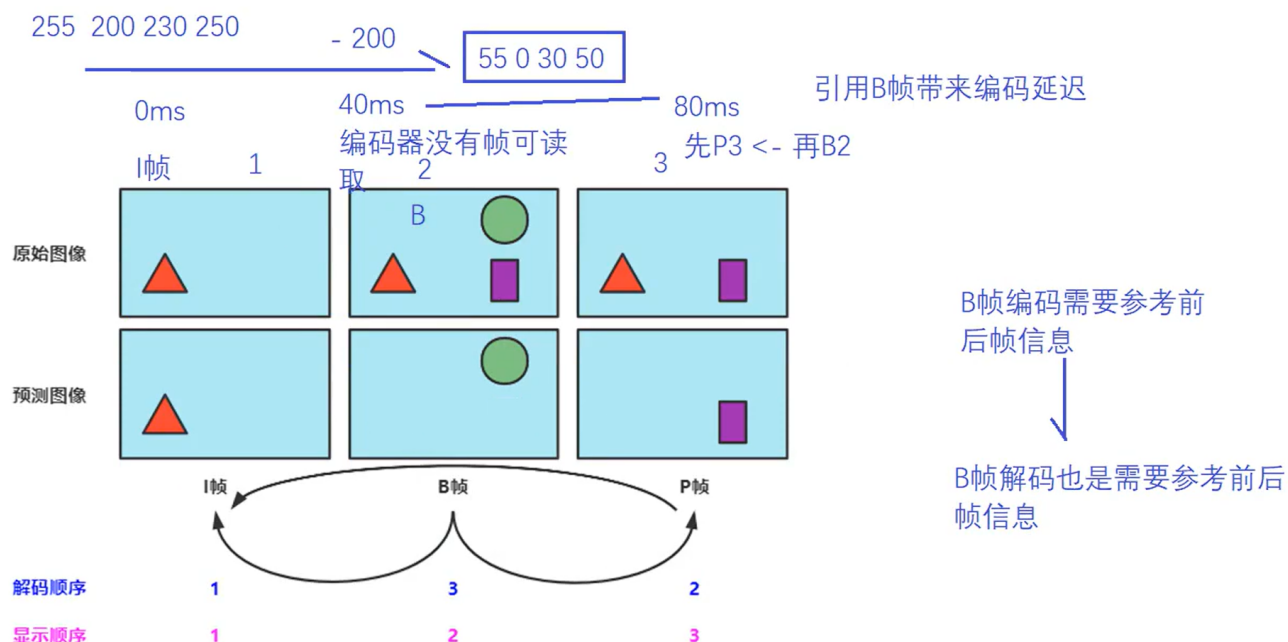


8.26 模拟面试

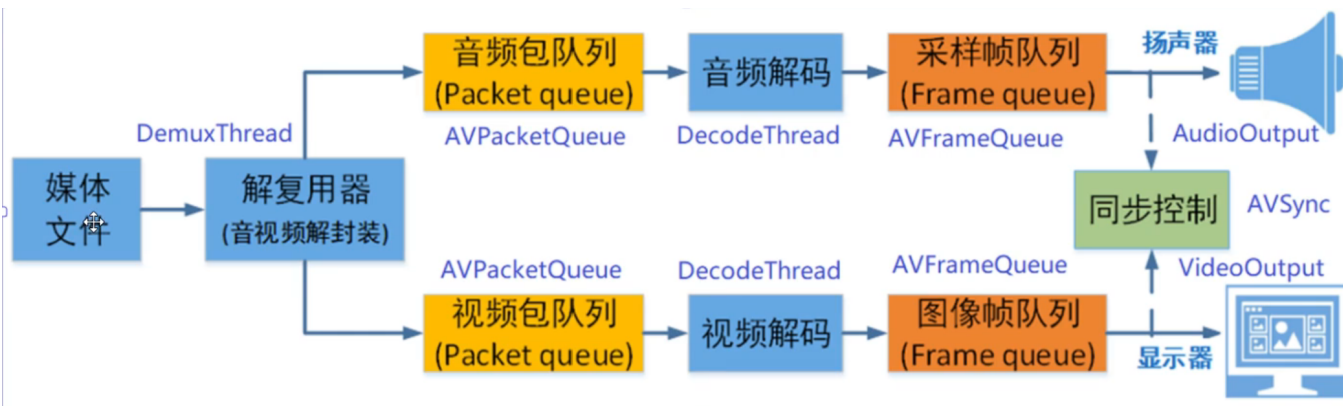
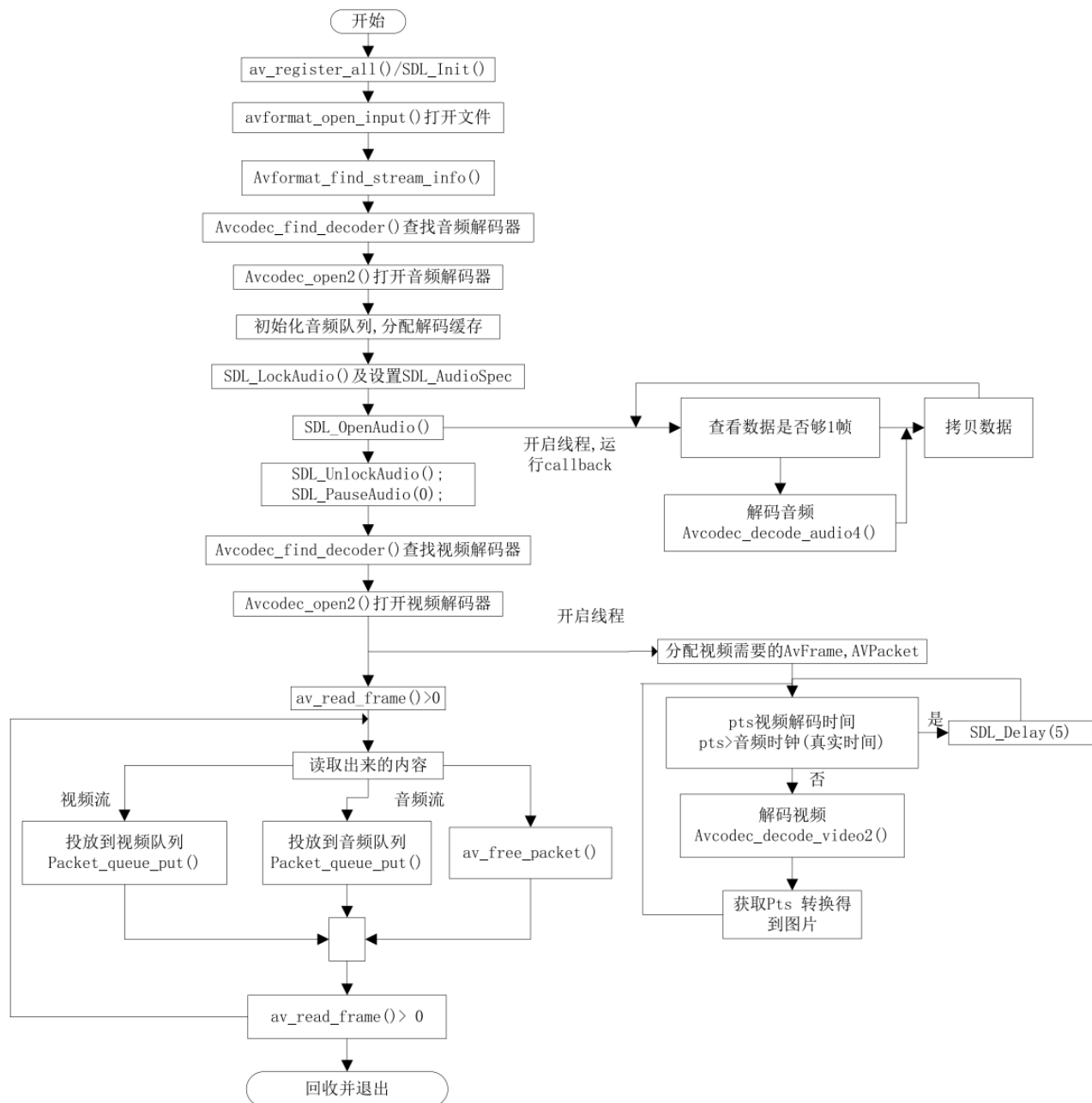
I帧 P帧 B帧

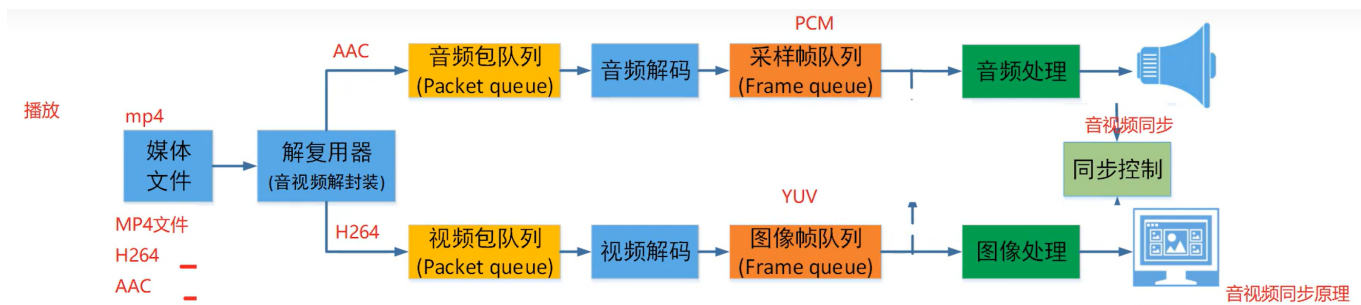
- I帧是帧内编码帧，本身具有完整的图像信息，不需要参考其他帧，因此在I帧可以切换频道，不会丢失图像而导致无法解码；
- P帧是 帧间编码帧，预测编码图像，需要利用 之前的I帧或者P帧进行预测编码；
- B帧是 帧间编码帧，双向预测编码图像，利用之前或之后 的I帧或P帧进行双向预测编码，压缩率高，不适合作为参考帧；虽然压缩率高但是需要更多的缓冲时间和更高的CPU占用率去 解码，因此适合本地存储以及视频点播，而不适合实时性的直播

B帧会带来编码解码延迟



讲一下解码的流程(怎么使用ffmpeg库)





1. 网络层：把 HLS 切片拉到本地

“首先是**网络层**。我们采用 Nginx-http-hls-module 做切片分发，客户端用 FFmpeg 的 `libavformat/hls.c` 解析 m3u8。它会自动把 `#EXTINF` 里的 ts 切片通过 HTTP 拉下来，内部维护一个 `AVIOContext` → `URLProtocol(http)` 的抽象，这样后续的解复用器就可以像读本地文件一样读网络流。”

2. 解复用：拆出 ts 流

“数据到了之后，**解复用层**调用

`avformat_open_input()` → `avformat_find_stream_info()` 拿到 `AVFormatContext`;

根据 `find_stream_index` 获取对应的音、视频流索引，再 `av_read_frame()` 循环取出 `AVPacket`，这是**原始的压缩数据** (H.264/AAC)，放入自定义的 `PacketQueue` 中。”

3. 解码：送到对应的解码线程用对应解码器解码

“接着进入**解码层**。

- 视频：H264→YUV 根据视频流对应的解码器上下文，根据上下文的 `codec_id` 使用 `avcodec_find_decoder(AV_CODEC_ID_H264)` 找到解码器并打开 `avcodec_open2`，创建视频解码线程进行解码 `avcodec_decode_video2`，拿到 `AVFrame` (YUV)。
- 音频：AAC→PCM 同理需要找到解码器并打开，但是需要多一步设置音频信息，用于打开音频设备，在自带的 `audio_callback()` 解码回调函数中进行解码 `avcodec_decode_audio4`，获取 `AVFrame`(PCM)。

4. 在解码线程 `video_thread` 中，将YUV数据发送给OpenGL之前进行时间同步；
5. 渲染层：送显/送声

“最后渲染层：

- 视频帧交给 OpenGL做 YUV→RGB→Texture；
- 音频帧交给 在主线程 `run()` 函数中调用 `SDL_PauseAudioDevice(audioID,0)` 函数消暂停，即开始播放，`SDL_OpenAudioDevice` 成功后，设备默认处于 暂停状态，必须调用此函数才会开始回调取数据；

什么是软解码，什么是硬解码？

硬解码：使用**专用硬件模块**来完成视频解码任务。 eg:

- Intel Quick Sync Video (核显)
- NVIDIA NVDEC/NVENC (独显)
- AMD VCE/UVD (A卡)
- 手机上的 SoC 中的 VPU (视频处理单元)

优点：解码速度快，CPU不参与或很少参与；

缺点:不是支持所有格式，eg AV1；依赖显卡驱动；不容易调试；

软解码：使用**CPU** (通用处理器) 通过软件算法来解码视频。 eg:

- FFmpeg (libavcodec)
- VLC、PotPlayer、MPV 等播放器内置解码器
- Chrome、Edge 浏览器内置解码器

优点：理论上支持所有格式，只要编码器更新就能解；灵活性高，可自定义解码参数，适合调试、转码、滤镜处理；跨平台，不依赖特定硬件，任何设备只要有CPU就能运行。

缺点：PU占用高，播放高码率视频可能卡顿。

11. 如何实现视频播放的跳转？

跳转操作可以直接使用 FFMPEG 的跳转函数 `av_seek_frame` 来实现，函数原型如下：

```
1  int av_seek_frame(AVFormatContext *s, int stream_index,
    int64_t timestamp, int flags);
```

参数名	类型	说明
s	AVFormatContext*	打开的媒体文件上下文（由 <code>avformat_open_input</code> 返回）
stream_index	int	指定流的索引，若为 <code>-1</code> ，则表示自动选择一个默认流（通常是主视频流）
timestamp	int64_t	目标时间戳，单位是 <code>AVStream.time_base</code> （即对应流的时间基）
flags	int	控制定位行为的标志位，常用的有以下几种：
标志位宏名	含义	
AVSEEK_FLAG_BACKWARD	定位到小于或等于 <code>timestamp</code> 的关键帧	
AVSEEK_FLAG_ANY	可以定位到任意帧（不一定是关键帧）	
AVSEEK_FLAG_FRAME	<code>timestamp</code> 以“帧数”为单位，而不是以时间戳为单位	

参数名	类型	说明
AVSEEK_FLAG_BYTE	timestamp 以字节为单位（慎用，效率低）	

- 不能直接简单的将当前 帧开始放入音、视频队列中进行处理，因为音视频队列中还有之前的数据，这些数据还可以播放几秒，因此首先需要清理 队列中这些数据
- 直接这样处理，会导致跳转时的花屏现象

原因：解码器中还保留了上一帧的信息，跳转后，会对当前帧产生影响，因此还需要清理解码器中的数据；

实现：往队列中放入一个特殊的 packet `#define FLUSH_DATA "FLUSH"`，当解码线程取到这个 packet 的时候，就执行清除解码器的数据

- 关键帧（I帧）问题：由于视频解码需要关键帧才能解码出有效信息，因此跳转时，是跳转到关键帧；

例如 10——15s之间，关键帧是第 10s 和第15s,如果要跳转到 第13s秒开始播放，实际上是跳转到第 10s，将第10——12s的数据扔掉，播放第13s的数据；

在run函数的循环中，处理跳转标志时，`av_seek_frame` 只能跳转到关键帧，（这里就会存在误差）同时向音、视频缓冲队列放入清除的包 `FLUSH`；

为了解决 `av_seek_frame` 的跳转误差，因此需要在音视频各自的解码线程中解决关键帧但未到目的帧的帧舍弃问题；

注意：并且在处理清除包 `FLUSH` 时一定要将音频时钟 `video_clock` 置0，否则 `video_clock` 一直记录的时跳转之前的时间 `00:05:00`，在经过同步算法 `synchronize_video` 后 pts 也与 `video_clock` 保持了一致是 `00:05:00.xx`，音频时钟此时才 `00:01:30.xx`，差距巨大 → **音视频同步算法会误以为要丢 3 分多钟的帧**；画面直接黑屏或出现长时间“等待音频追赶”。

关键帧：不依赖其他帧即可独立解码的视频帧；也就是说I帧本身就是完整的画面，无需再参考前后帧再解析；

关键：

- 解码的起点：播放器必须先拿到一个I帧才能开始正确解码后序P/B帧
- 跳转：会找最近的I帧开始解码
- 错误恢复：网络包丢失or损坏后，下一个I帧可以重置画面，防止错误累积；